

第一章 命令行、Shell 与本地帮助

命令行题首先考察的不是输入速度，而是 Shell 如何把字符转换为命令、参数和数据流。引号、展开、重定向或退出状态理解错误，会让一条看似正确的命令作用于错误对象。本章把命令解析、输入输出、本地帮助和安全组合命令组织成可验证的操作方法。

[概念] Shell 是读取命令行、完成展开和重定向、再启动命令的解释环境。命令名、选项和位置参数只是常见约定；真正支持哪些参数，应以本机帮助和手册为准。

[概念] 标准输入（stdin，文件描述符 0）、标准输出（stdout，1）和标准错误（stderr，2）是进程默认的数据通道。管道只把前一命令的 stdout 连接到后一命令的 stdin，stderr 不会自动进入管道。

[操作语义] `type` 判断名称如何被 Shell 解析；`man`、`--help`、`apropos` 和 `info` 从不同入口查询本机用法；`dnf provides` 从文件路径反查软件包。

[操作语义] `>` 重建输出文件，`>>` 追加，`2>` 单独重定向错误，`2>&1` 让 stderr 指向 stdout 当前目标；`tee` 在管道中同时写文件并继续输出。

[知识专题] Shell 解析、引用与展开顺序

① [知识点] 一条命令先由 Shell 解释

Shell 先识别引号、变量、命令替换、通配符和重定向，再把结果作为参数交给程序。空格通常分隔参数，因此含空格的路径必须引用；通配符由 Shell 展开为现有文件名，程序通常看不到原始的 `*`。

```
printf '%s\n' "$HOME"      # 双引号内展开变量
printf '%s\n' '$HOME'     # 单引号保留字面文本
printf '%s\n' './*.conf'  # Shell 展开匹配文件
```

双引号保留变量和命令替换，但抑制分词与文件名展开；单引号几乎逐字保留内容。需要在单引号文本中出现单引号时，应结束引用、转义该字符、再重新开始，而不是不断增加反斜线猜测结果。

② [知识点] 命令替换与退出状态回答不同问题

`$(command)` 把命令 stdout 去掉末尾换行后嵌入当前命令；`$?` 是上一条前台命令的退出状态。状态 `0` 通常表示该命令按自身定义成功，非零表示不同类型失败，但 `0` 不能证明整个业务目标完成。

```
today=$(date +%F)          # 捕获标准输出
tar -cjf "backup-{$today}.tar.bz2" /srv # 使用替换结果
printf 'exit=%s\n' "$?"    # 紧接着读取退出状态
```

读取 `$?` 前若先运行了其他命令，得到的就是后一命令状态。需要基于成功与否继续执行时，`cmd1 && cmd2` 只在前者成功后执行后者；`cmd1 || cmd2` 只在前者失败后执行后者。分号无条件继续，不能表达

验证门。

③ [边界] 变量必须明确是否允许分词

路径、用户输入和命令替换结果通常应使用 "\$var"。未引用变量会经历分词和文件名展开，空格或 * 可能改变参数数量。不要把命令及参数整体塞进字符串再依赖未引用展开；复杂命令应直接书写，脚本需要动态参数时使用数组。

[Cheatsheet] 字面文本用单引号；需要变量展开用双引号；命令输出用 \$(...)；上一命令状态用 \$?；成功链用 &&，失败分支用 ||。

[操作专题] 精确控制 stdout、stderr 与管道

① [参数点] 覆盖、追加和错误通道不能混淆

> 在命令启动前打开并截断目标文件，即使命令随后失败，旧内容也可能已经丢失。>> 追加到末尾。数字 2 明确选择 stderr；没有数字默认处理 stdout。

```
find /etc -name '*.conf' > /tmp/files.txt          # 覆盖 stdout
find /etc -name '*.conf' 2> /tmp/find-errors.txt    # 单收 stderr
date >> /tmp/run.log                                # 追加 stdout
```

合并两个通道时，顺序决定含义：>file 2>&1 先让 stdout 指向文件，再让 stderr 复制该目标；2>&1 >file 先让 stderr 指向当时的终端，随后只改变 stdout，因此不会把两者都写入文件。

② [操作点] 管道只传递可消费的数据

管道适合把稳定文本交给筛选或统计工具。前一命令的错误默认仍显示在终端。如果题意要求错误也进入后续命令，可显式使用 2>&1 |；但排错时把错误混入数据会污染结果，通常应分别保存。

```
journalctl -u sshd --since today | grep -i 'fail' | wc -l # 日志筛选与计数
printf '%s\n' alpha beta | tee /tmp/items | sort           # 写文件并继续传递
```

tee -a 追加而不是覆盖。需要 root 权限写受保护文件时，sudo command > /root/file 的重定向仍由当前 Shell 执行；可让具有权限的 tee 接收管道数据，但必须确认覆盖或追加语义。

③ [验证点] 检查文件内容、通道和状态

用 wc -l 或 sed -n 检查结果规模与代表性内容，用 stat 确认目标文件，用单独错误文件确认是否遗漏权限失败。管道默认的 \$? 是最后一个命令状态；交互排错可查看 Bash 的 PIPESTATUS 数组，脚本需要整条管道任一环节失败时可考虑 set -o pipefail。

[Cheatsheet] stdout: > / >> ; stderr: 2> / 2>> ; 合并到同一目标: >file 2>&1 ; 管道只接 stdout; 边看边存: tee , 追加用 tee -a 。

[操作专题] 从目标反查命令、参数与软件包

① [操作点] 先识别当前名称是什么

`type` 是 Shell 语义入口，能区分 alias、关键字、函数、内建命令和外部可执行文件。`which` 主要按 PATH 查可执行文件，无法完整解释别名或 Shell 内建；`command -V` 也能给出解析结果。

```
type cd                # Shell 内建
type -a ls             # 列出所有可解析入口
command -V useradd     # 描述命令解析结果
```

② [操作点] 按由近到远的帮助入口查询

已知命令时先看 `command --help` 的短语法，再用 `man command` 查完整说明。Shell 内建可用 `help <builtin>`；配置文件常有独立手册，如 `man 5 passwd`。`man` 的 section 区分普通命令、系统调用、库、设备、文件格式和管理命令。

```
useradd --help        # 短选项入口
man useradd           # 完整命令手册
man 5 crontab         # 文件格式手册
help test             # Bash 内建帮助
```

不知道命令名时，先确保 `mandb` 索引可用，再用 `apropos 'password aging'` 或 `man -k` 搜索描述。`info` 对部分 GNU 工具有更完整的主题式说明。

③ [操作点] 命令缺失时反查包

先确认不是 PATH、拼写或别名问题。若确实缺少文件，用路径模式查询仓库：

```
dnf provides '*/semanage'    # 按可执行文件名反查包
dnf provides /usr/bin/rsync  # 已知完整路径时精确查询
```

查询结果只证明仓库元数据中哪个包提供文件；还要确认仓库可用、安装正确包，并重新用 `type` 或 `rpm -qf` 验证。

④ [验证点] 用帮助得到最小可执行答案

从手册中提取本题需要的命令骨架、关键参数、对象位置和验证入口，不抄参数大全。运行前可用 `--help` 复核大小写和增量符号；运行后仍按对象状态验收，帮助页不会证明配置已生效。

[Cheatsheet] 名称解析：`type -a`；短帮助：`--help / help`；完整手册：`man`；按描述找命令：`apropos`；按文件找包：`dnf provides '*/name'`。

[诊断专题] 从空结果、错误文件和退出状态推进证据

① [诊断点] 空输出不一定是成功答案

`grep` 没有输出可能表示无匹配，也可能上游文件不可读；`find` 结果不全可能伴随 `stderr` 的 `Permission denied`；通配符未匹配时在默认 `Bash` 配置下可能原样传给程序。先查看退出状态和错误通道，再解释空结果。

② [诊断点] “command not found” 先解析与安装

检查拼写、`type -a`、`echo "$PATH"` 和绝对路径。只有确认命令文件不存在后，才使用 `dnf provides`。不要为了一个命令无条件安装猜测的软件包，也不要当前目录加入 `PATH` 作为通用修复。

③ [诊断点] 文件被清空先检查重定向时机

`sort file > file` 会在 `sort` 读取前截断同一文件。应写入临时文件，验证后原子替换，或使用工具支持的原地模式。看到零字节文件时，不要重复相同命令，应先保留现场和确认是否有备份。

[Cheatsheet] 空结果查退出状态与 `stderr`；找不到命令查 `type /PATH` 再查包；输入输出是同一文件时禁止直接 `> 原文件`；重复执行前先判断 `Shell` 已经改变了什么。

[经典任务] 在陌生系统中找到归档命令并保存可审计结果

系统需要从 `/etc` 中找出名称以 `.conf` 结尾的普通文件，将可读取的路径排序后保存到 `/root/conf-files.txt`，把权限错误单独保存到 `/root/conf-errors.txt`，并创建日期为当天的 bzip2 归档 `/root/conf-YYYY-MM-DD.tar.bz2`。

你不确定归档工具的压缩参数，也不确定相关命令是否已安装。不得把错误信息混入路径列表，不得因为某些文件不可读而覆盖已有归档；最终要证明路径列表、错误文件、归档格式和归档内容分别符合要求。

卡住时再想一想：哪一通道进入排序？日期怎样安全嵌入文件名？帮助和软件包查询分别解决什么问题？

请先独立完成。参考解答从下一页开始。

[参考解答] 分离数据与错误，再按帮助构造归档

① [操作点] 调查命令与目标文件

先用 `type -a find sort tar date` 确认解析入口，用 `tar --help` 或 `man tar` 查 `-c`、`-j`、`-f` 的语义。若工具缺失，再执行 `dnf provides '*/tar'`，不要从记忆猜包。

② [操作点] 分离输出并建立日期变量

`find` 的 stdout 进入 `sort`，stderr 单独写入错误文件。先写临时路径列表，验证后再替换正式文件，避免失败时覆盖已有结果。

```
today=$(date +%F) # 生成 ISO 日期
find /etc -type f -name '*.conf' \
  2> /root/conf-errors.txt | sort > /root/conf-files.txt.tmp
test -s /root/conf-files.txt.tmp && \
  mv /root/conf-files.txt.tmp /root/conf-files.txt # 有结果才替换
```

③ [操作点] 依据列表创建归档

`tar -T` 从文件读取成员列表；`-c` 创建，`-j` 使用 bzip2，`-f` 后紧跟归档名。

```
tar -cjf "/root/conf-${today}.tar.bz2" \
  -T /root/conf-files.txt # 按路径清单归档
```

④ [验证点] 分别证明四个对象

```
sed -n '1,10p' /root/conf-files.txt # 抽查排序路径
wc -l /root/conf-files.txt /root/conf-errors.txt # 结果与错误规模
file "/root/conf-${today}.tar.bz2" # 归档格式
tar -tjf "/root/conf-${today}.tar.bz2" | sed -n '1,10p' # 成员抽查
```

退出状态只说明对应命令结果；路径列表、错误通道、压缩格式和成员内容需要分别检查。

[Cheatsheet] `type` / 帮助确认语法 → stdout 排序、stderr 分离 → `date +%F` → `tar -cjf ... -T list` → `wc + file + tar -tjf` 分层验收。

[本章收束] 把字符写法还原为参数和数据流

Shell 题的核心是知道谁在解释字符。引号和展开决定程序实际收到哪些参数；重定向和管道决定数据流向哪里；退出状态只描述一条命令或管道的局部结果；本地帮助提供准确语法，却不能替代操作后的对象验证。

面对陌生命令，先用 `type` 确认解析，再从短帮助、手册、描述搜索和包反查逐步靠近答案。面对空输出或错误，先分离 `stdout` 与 `stderr`，并保留证据。这样命令行就不是符号记忆，而是一套可推演、可验证的接口。